Repaso de IG con Unity (2)

Hasta ahora ya sé...

Marco Antonio Gómez Martín

Iluminación y Materiales

Grado en Desarrollo de Videojuegos

2025/2026

Facultad de Informática

UNIVERSIDAD COMPLUTENSE MADRID

Semántica

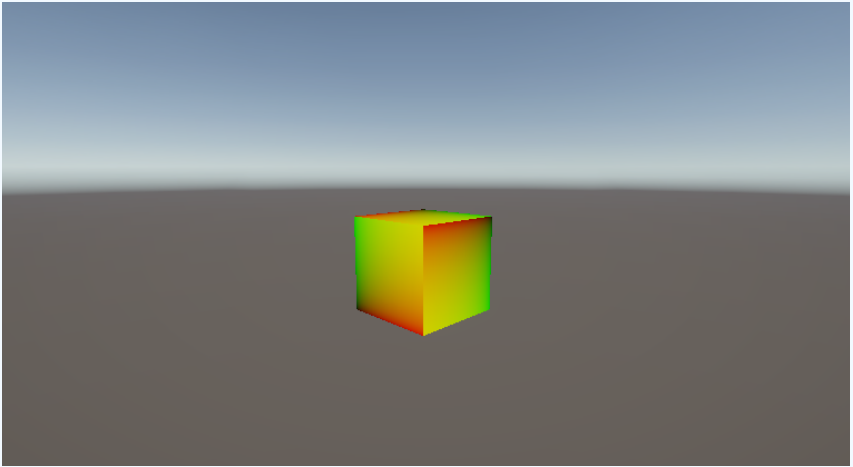
La semántica que aparece en estructuras y valores devueltos, ¿a quién van dirigidas?

- Motor
- Hardware
- Programadores de shaders

Interpolación

```
struct VsIn {
    float4 vertex : POSITION;
    float2 texcoord : TEXCOORD0;
};
struct VsOut {
    float4 pos : SV_POSITION;
    float2 uv : TEXCOORD0;
};
VsOut vsMain(VsIn v) {
    VsOut o;
    o.pos = TransformObjectToHClip(v.vertex.xyz);
    o.uv = v.texcoord;
    return o;
}
float4 psMain(VsOut i) : SV_TARGET {
    return float4(i.uv.x, i.uv.y, 0.0, 1.0);
}
```

Interpolación



Interpolación

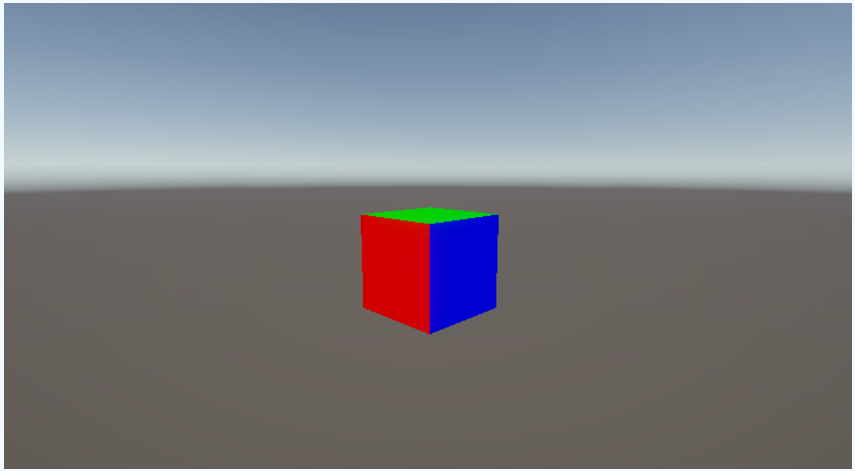
```
struct VsIn {
    float4 vertex : POSITION;
    float3 normal : NORMAL;
};

struct VsOut {
    float4 pos : SV_POSITION;
    float4 color : COLOR;
};

VsOut vsMain(VsIn v) {
    VsOut o;
    o.pos = TransformObjectToHClip(v.vertex.xyz);
    o.color = float4(abs(v.normal), 1.0);
    return o;
}

float4 psMain(VsOut i) : SV_TARGET {
    return i.color;
}
```

Interpolación



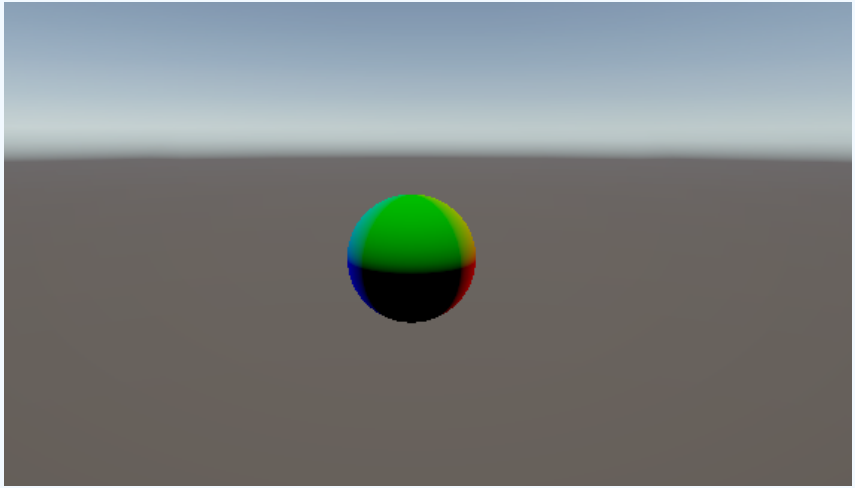
Interpolación

- ¿Por qué `abs(v.normal)`?
- ¿Alguna cosa mejorable / incorrecta?

```
VsOut vsMain(VsIn v) {
    VsOut o;
    o.pos = TransformObjectToHClip(v.vertex.xyz);
    o.color = float4(abs(v.normal), 1.0);
    return o;
}
float4 psMain(VsOut i) : SV_TARGET {
    return i.color;
}
```

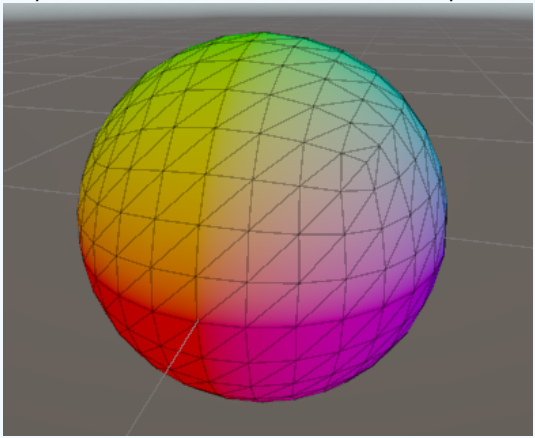
Interpolación

Esfera sin valor absoluto y normalizando



Interpolación

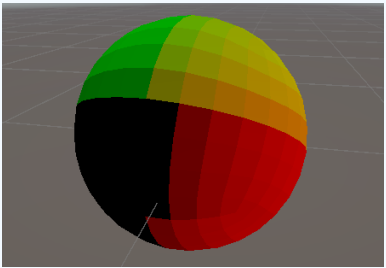
Interpolación de normales en cada primitiva



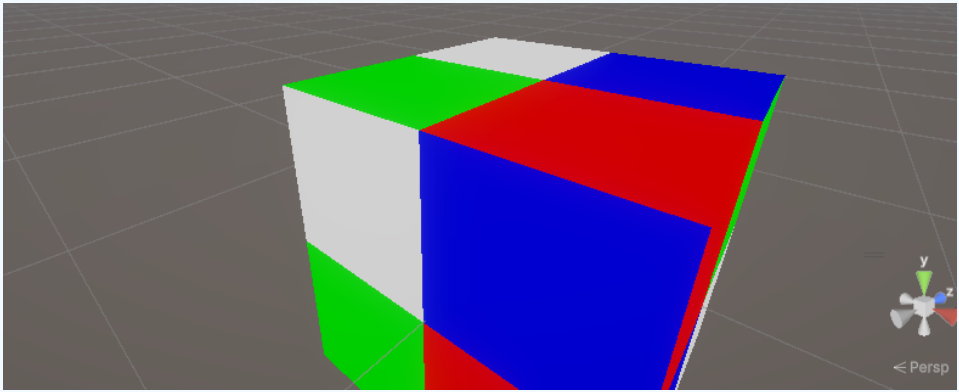
¿Y si no interpolamos?

Interpolación

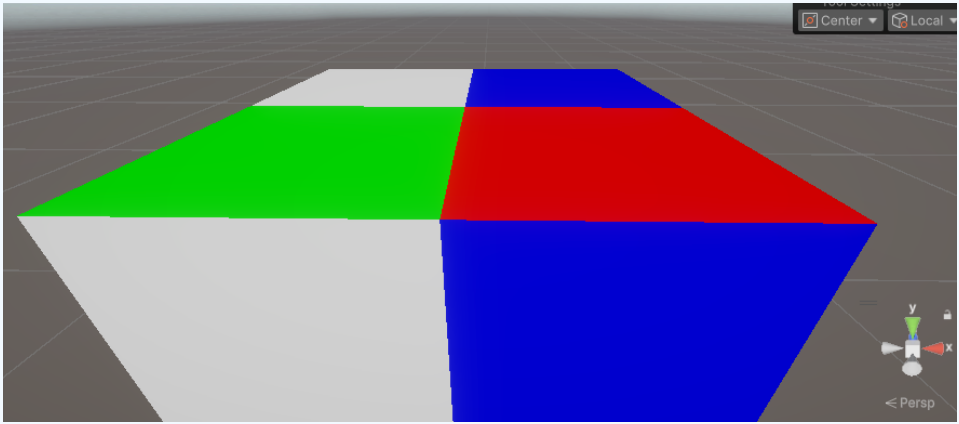
```
struct VsOut {
  float4 pos : SV_POSITION;
  nointerpolation float4 color : COLOR;
};
```



Corrección de perspectiva



Corrección de perspectiva



Corrección de perspectiva

Next we must specify how the data associated with each rasterized fragment are obtained. Let the window coordinates of a produced fragment center be given by $\mathbf{p}_r = (x_d, y_d)$ and let $\mathbf{p}_a = (x_a, y_a)$ and $\mathbf{p}_b = (x_b, y_b)$. Set

$$t = \frac{(\mathbf{p}_r - \mathbf{p}_a) \cdot (\mathbf{p}_b - \mathbf{p}_a)}{\|\mathbf{p}_b - \mathbf{p}_a\|^2}. \tag{14.5}$$

(Note that $t = 0$ at $\mathbf{p}_a$ and $t = 1$ at $\mathbf{p}_b$). The value of an associated datum f for the fragment, whether it be a shader output or the clip w coordinate, is found as

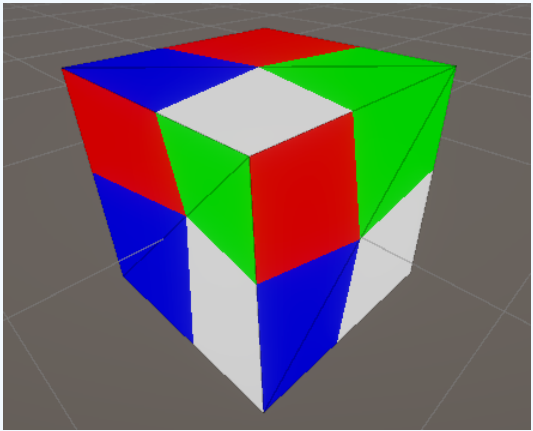
$$f = \frac{(1-t)f_a/w_a + tf_b/w_b}{(1-t)/w_a + t/w_b} \tag{14.6}$$

where f_a and f_b are the data associated with the starting and ending endpoints of the segment, respectively; w_a and w_b are the clip w coordinates of the starting and ending endpoints of the segments, respectively. However, depth values for lines must be interpolated by

$$z = (1-t)z_a + tz_b \tag{14.7}$$

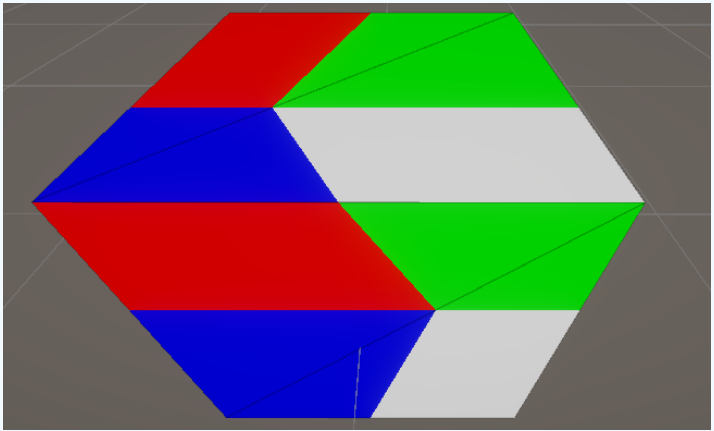
where z_a and z_b are the depth values of the starting and ending endpoints of the segment, respectively.

Corrección de perspectiva



`noperspective`

Corrección de perspectiva

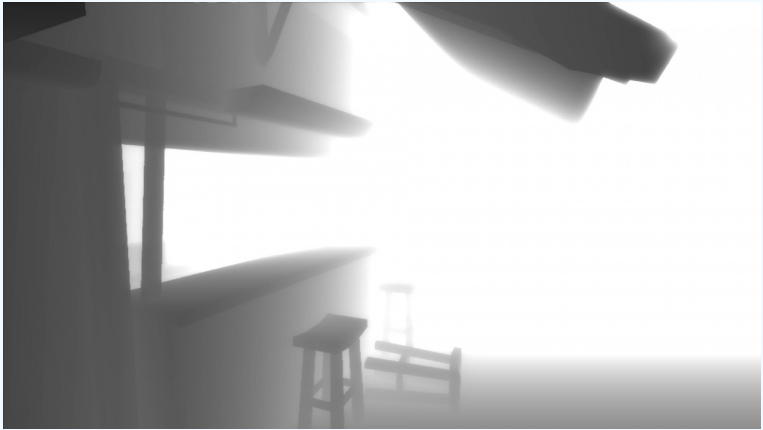


`noperspective`

Z-buffer

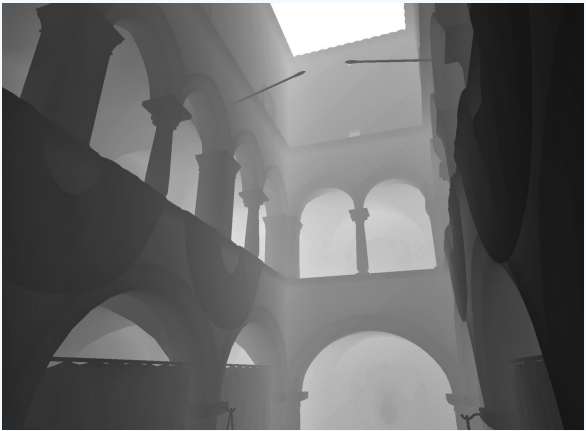
- Qué es y para qué sirve
- Qué valores guarda
- Z-test en los shaders

Ejemplo z-buffer

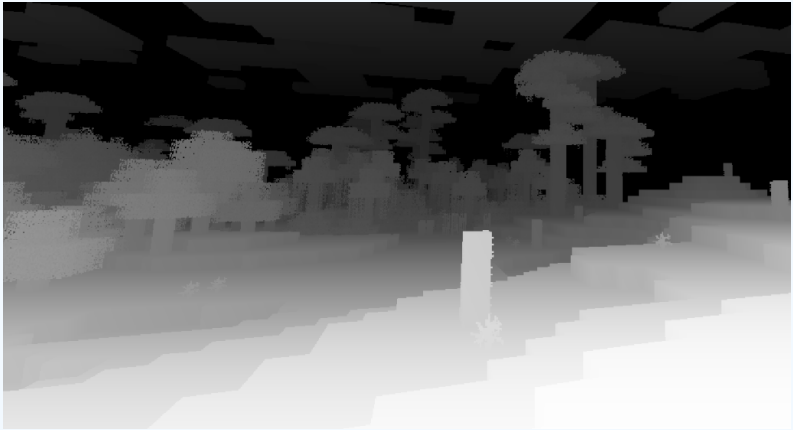


Ejemplo z-buffer

Ejemplo z-buffer



Ejemplo z-buffer

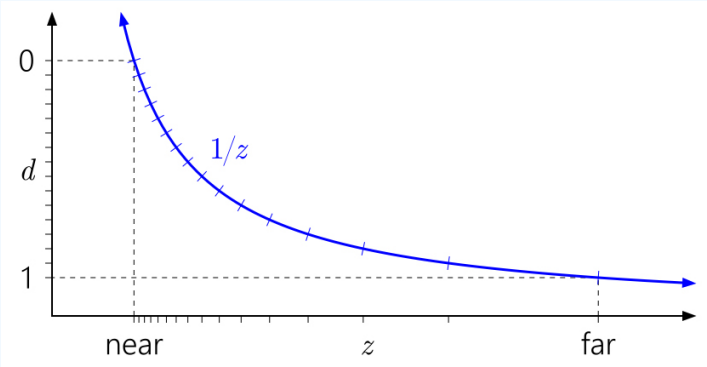


Z-buffer - Valores

- Distribución *no lineal* de valores.
- En Unity por defecto, plano cercano en 0.3 y lejano en 1000.
- El 70 % del espacio (entre 0 y 0.7) representa el primer metro de distancia.
- Si el ratio plano lejano / plano cercano es 100, el 90 % del rango se gasta en el 10 % del espacio inicial. Con 1000, el 98 % es el primero 2 %.

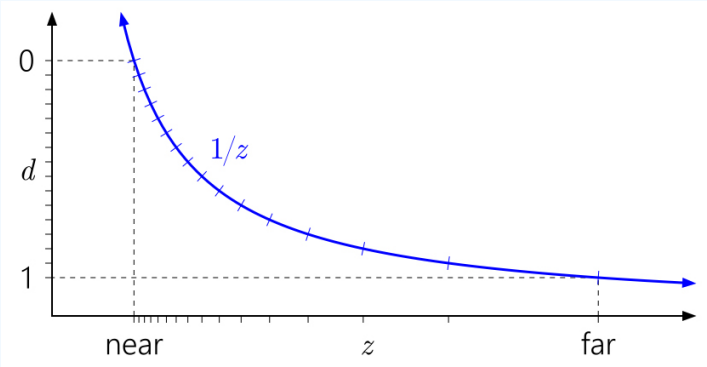
Z-buffer - Valores

https://developer.nvidia.com/content/depth-precision-visualized



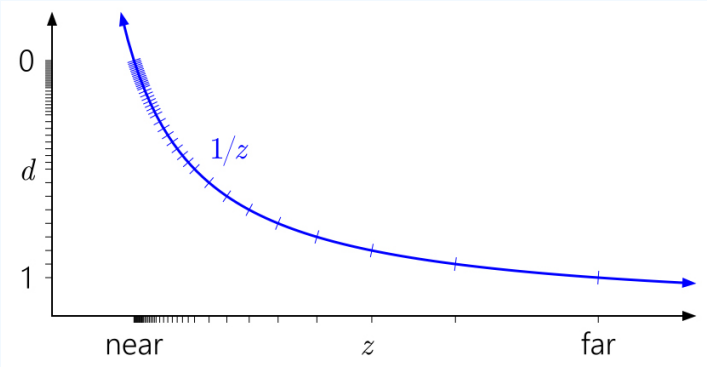
Z-buffer - Valores

https://developer.nvidia.com/content/depth-precision-visualized



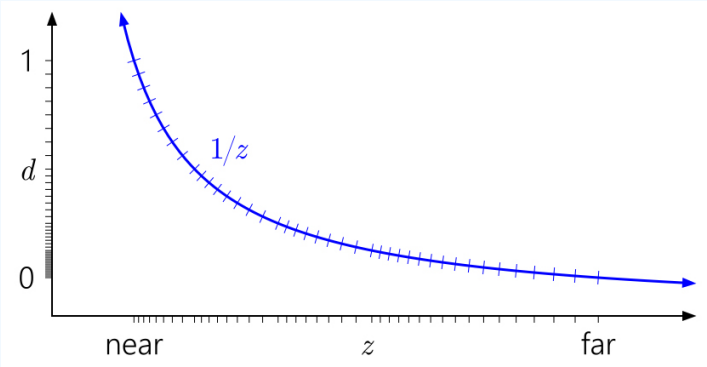
Z-buffer - Valores

https://developer.nvidia.com/content/depth-precision-visualized



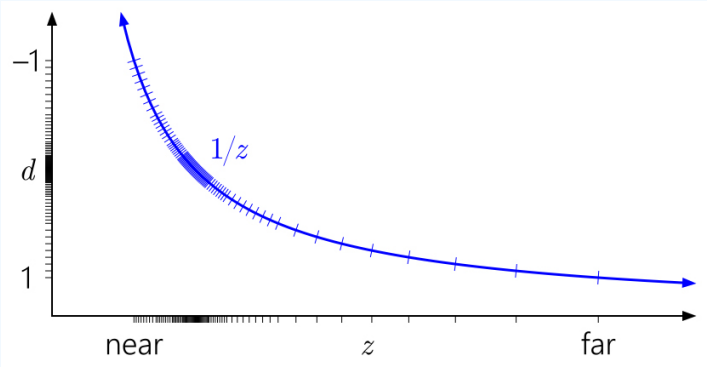
Z-buffer - Valores

https://developer.nvidia.com/content/depth-precision-visualized



Z-buffer - Valores

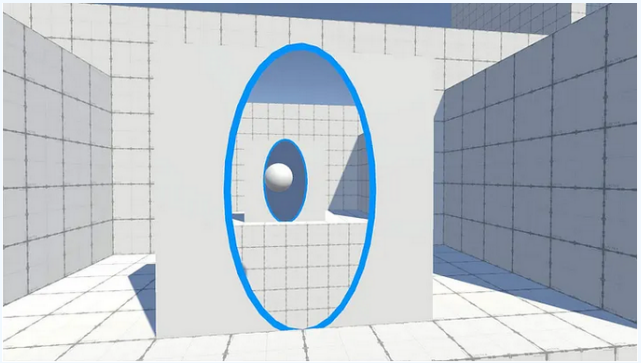
https://developer.nvidia.com/content/depth-precision-visualized



Z-buffer: optimizaciones

- *early depth testing*
- *z-pre pass*

Stencil buffer



Stencil buffer

- Valores más pequeños, enteros (normalmente 8 bits).
- Se compara con un valor fijo para toda la primitiva.
- Test similar al z-buffer (LEqual, Less, GEqual, Greater, Equal, NotEqual, Always, Never).
- Se especifica qué hacer con el buffer tanto si pasa como si no.
- Keep, Zero, Replace, IncrSat/DecrSat, Invert, IncrWrap/DecrWrap.

Repaso de IG con Unity (2)

Hasta ahora ya sé...

¿Preguntas?

Facultad
de
Informática

UNIVERSIDAD COMPLUTENSE
MADRID